FSDS · GEN AI & AGENTIC AI · CLASS NOTES

LLM Application Development Part 2

Vector stores, retrievers, LangChain chains, memory, agents, PDF loading and an introduction to LLM fine-tuning, with hands-on practicals.

Vector Stores | Retrievers | Sequential Chains | Memory | Agents | Fine-tuning

Prepared for Roja · Session notes & lab guide

Contents

Theory sections are in indigo; hands-on labs are in teal.

1. Where we are: the LLM app pipeline

2. Vector stores

3. Retrievers and vector search

4. LangChain chains

Lab 1 · Simple sequential chain

5. LangChain memory

Lab 2 · Conversation memory

6. LangChain agents

Lab 3 · Math agent (and Wikipedia)

Lab 4 · Loading a PDF

7. LLM fine-tuning

Lab 5 · Preparing data for fine-tuning

8. Quick revision & what's next

RECAP

1 Where we are: the LLM app pipeline

In the last session we built the first half of a Retrieval-Augmented Generation (RAG) pipeline: loading documents, splitting them, embedding them and saving them in a FAISS vector store. This session covers **how the store works internally** and the LangChain pieces that connect everything: **chains, memory and agents**. It ends with an introduction to **fine-tuning**.

Loader → Splitter → Embedding → Vector Store → Retriever → Prompt + LLM → Output parser

BIG IDEA

LangChain is the "connector" that joins these systems together. Chains link the steps, memory remembers context between calls, and agents choose which tools to use.

Agenda for this module

- **Vector stores:** what they are and why data is stored as vectors.
- **Retrievers:** how a prompt pulls the right information out of the store.
- **Chains:** the sequential chain plus the Stuff, Refine and MapReduce chains.
- **Memory:** how a conversation remembers earlier messages.
- **Agents:** LLMs that use external tools.
- **Fine-tuning:** adapting an LLM to a specific domain.

THEORY

2 Vector stores

A **vector store** is a storage space for embeddings. Every piece of data, whether text, images, PDFs or social-media posts, is converted into the **same numeric (vector) format** before it is saved.

COMMON CONFUSION

A vector store is **not temporary memory**. It is the main, persistent storage of your data. Its size is limited only by the disk you give it (for example 5 TB on an offline server).

Why convert everything to vectors?

- **One format, one system.** If all data is in one format, a single retrieval system can search it, which is easier to optimise.
- **Space.** A 45 MB image becomes a few KB of numbers. Training data runs into lakhs of items, so storing raw files would need enormous infrastructure (think of Facebook's warehouse-sized data centres).
- **Speed.** LLMs process embedded vectors directly and efficiently.

> **ANALOGY: THE COMMON FUEL**
>
> If Maruti, BMW and Audi each needed a different fuel, drivers would struggle to find pumps. Makers built compatible engines so that any pump works. Vectors are that "common fuel": every source is converted to one format, and every query is converted to the same format.

The ETL view

Extract (any source) → Transform + Embed → Load into vector store

Storing and removing

Stores can add new items and retire old ones. When information goes out of date (for example data on keypad Nokia phones), it is **archived** (often as compact JSON) rather than deleted. A query checks the main store first and falls back to the archive.

Popular implementations

Store	Notes
FAISS (Facebook AI)	Used in our lab. Fast and handles large, high-dimensional data.
Chroma	Simple and popular. The lab's version is outdated, which is why we used FAISS.
PGVector	A vector extension for PostgreSQL.
Pinecone	A managed cloud vector database.

THEORY

3 Retrievers and vector search

A **retriever** fetches relevant data from the vector store. It uses **similarity search**: the query is embedded, compared with the stored vectors, and the closest matches are returned.

Prompt → Embed query → Similarity search in store → Top matches → LLM generates answer

> **ANALOGY: BUYING A LAPTOP**
>
> Your query ("15.4-inch screen, 16 GB RAM...") is matched against every laptop's specs in the store. Only the matching ones come back. Similarity search is filtering, except it works on vectors instead of raw data.

Key points

- **Embedding works both ways:** it converts data into vectors when storing and is used again when interpreting the retrieved results. It relies on tokenization plus heavy mathematics (similarity and probability).
- **Retrievers only "speak vector store".** The store is the backbone of retrieval. Simpler retrievers exist for small data, such as searching folders.

- **Multiple equal matches:** you get several options, which is why ChatGPT sometimes offers alternative answers. With one answer required, chat history and preferred style decide which one wins.
- The `similarity_search()` call we used last class **is** a retriever in action.

Load → Transform → Embed → Store → Retrieve

Load → Transform is ordinary data pre-processing, the same idea as in ML/DL.

THEORY

4 LangChain chains

An LLM alone handles single tasks well. Real applications need **multiple stages**, each with a different purpose. **Chains** are the connectors that link these stages, like the USB cable between a keyboard and a laptop.

CHAIN INTERFACE (LEGACY)
Like a wired keyboard: a direct, explicit connection. Used for **simpler** systems and for learning.

LCEL
LangChain Expression Language, like Bluetooth: more flexible. Used for **complex, large** applications. Both are used in industry.

LLMChain: the foundation

An `LLMChain` combines a **prompt template** (input variables + template text) with a **language model** (LLM or chat model). A chain does nothing until a model or agent is attached and given input, just as `print()` needs a value.

Sequential chain

As in deep learning, where layer 1's output feeds layer 2, a **sequential chain** passes chain 1's output in as chain 2's input. Each chain has its own template and input variable. The LLM can be shared.

Input topic → Chain 1: write a sentence → Chain 2: create a title → Final output

Document chains for retrieval

Chain	How it works	Best for
Stuff	"Stuffs" the question and all retrieved context into one prompt. Builds candidate interpretations and sends the most similar to the LLM.	Small documents
Refine	Like Stuff, but adds document formatting and refines the answer step by step, "polishing" it with each document.	Higher-quality answers
MapReduce	Splits data into small pieces, processes each one (*map*), then combines the most relevant results (*reduce*). Avoids scanning the whole document.	Very large data

PRACTICAL

L1 Lab 1 · Simple sequential chain

Goal: take a topic, write a short sentence about it (chain 1), then create a title for that sentence (chain 2).

```python
from langchain.chat_models import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain, SimpleSequentialChain

# 1. The LLM (shared by both chains)
llm = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0)

# 2. Chain 1: write a short sentence on the topic
chain1 = LLMChain(
    llm=llm,
    prompt=PromptTemplate(
        input_variables=["topic"],
        template="Write one short sentence about {topic}."))

# 3. Chain 2: create a title for that sentence
chain2 = LLMChain(
    llm=llm,
    prompt=PromptTemplate(
        input_variables=["text"],
        template="Create a short title for this: {text}"))

# 4. Combine: list order = execution order
overall_chain = SimpleSequentialChain(chains=[chain1, chain2], verbose=True)

print(overall_chain.run("data science"))
# → "Unlocking Insights: The Power of Data Science"
```

EXPERIMENTS TO TRY

- Change the topic (*analytics*, *healthcare*, *fitness*). The domain changes both the sentence and the title.
- **Swap the order:** `chains=[chain2, chain1]` changes the whole output, because order decides the flow.
- **Reuse a chain:** `chains=[chain1, chain2, chain1]` runs chain 1 again as a third stage, with no need to define a "chain 3".
- Try other templates: "Give 3 keywords about {topic}" or "Give 3 important points about {topic}".

READING THE OUTPUT

With `verbose=True` you will see each chain's output separately. That separator shows where chain 1 ends and chain 2 begins. A third chain would add a third block.

VERSION NOTE

Newer LangChain releases deprecate `LLMChain` and `.run()`. The LCEL equivalent is: `chain = prompt1 | llm | StrOutputParser() | (lambda t: {"text": t}) | prompt2 | llm` then `chain.invoke({"topic": "data science"})`.

THEORY

5 LangChain memory

Memory is temporary space that stores your conversation so the LLM can use earlier messages. Your ChatGPT history works the same way: delete it and the model "forgets" your style and context.

Input → Memory ⇄ LLM → Output → back to Memory

Example: say "My name is Shivraj, I am a data science trainer." Later ask "What is my name?" and it answers from memory.

Types of memory

Type	What it stores
ConversationBufferMemory	The **entire** conversation, every message (used in our lab).
ConversationBufferWindowMemory	Only the last *k* exchanges, a sliding window of the chat.
ConversationEntityMemory	Only the key **entities**/facts (names, places, metadata), not the full text.
ConversationSummaryMemory	A running **summary**. Useful for "summarise our last month's chats".
VectorStoreRetrieverMemory	Stores memories in a **vector store** and retrieves the relevant ones by similarity.

The slides also mentioned ACM (Adaptive Case Management) for case-specific memory storage.

IN PRACTICE

A full application (for example a healthcare bot) can combine several memory types. More memory means more tokens used, but large projects need it.

PRACTICAL

L2 Lab 2 · Conversation memory

```python
from langchain.chat_models import ChatOpenAI
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory

llm = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0)
memory = ConversationBufferMemory()

# No prompt template needed, just the LLM + memory
chat = ConversationChain(llm=llm, memory=memory)

print(chat.predict(input="My name is John"))
# → Hello John, it's nice to meet you. How can I assist you today?
print(chat.predict(input="I am learning machine learning"))
print(chat.predict(input="What is my name?"))
# → Your name is John.

# View the saved history (like ChatGPT's sidebar)
print(memory.buffer)
```

WHY PREDICT()?

A machine needs a command. With no template, `predict()` tells the LLM to **generate a reply** to the input, just as `model.predict()` does in ML. That is why "My name is John" produces a greeting rather than an echo.

GOTCHA

`memory.buffer` shows history up to the moment you run it. After adding new inputs (for example "What is next to machine learning?"), run it again to see the new Human/AI turns.

THEORY

6 LangChain agents

An **agent** is an LLM that uses **external tools** to complete a specific task, such as a calculator, Wikipedia, Google search, YouTube, a Python REPL or a database. Agents are the "conductors" that run the process. Hugging Face hosts well over a million models and tools you can build on.

Task → Agent chooses action → Tool runs → Observe result → Repeat until done → Answer

- **Specialised beats general:** a math agent gives reliable calculations, and a PPT agent builds better decks than a basic chat.
- **Free vs paid:** free tiers often have limited agents that stop after a few tasks ("analytics not working, try later"). Paid tiers keep agents active.
- **Team analogy:** each agent is a team member with a dedicated role.
- **Sharing:** agents connected through the same chain share information. Separate agents in separate processes do not.

Agent recipe

Create LLM → Load tools → Initialise agent (type) → Run / invoke

The slides also show the newer pattern: `prompt = hub.pull("hwchase17/react")` → `create_react_agent(llm, tools, prompt)` → `AgentExecutor(agent=..., tools=..., verbose=True)`. Use `verbose=True` to see the agent's decision-making.

PRACTICAL

L3 Lab 3 · Math agent (and Wikipedia)

```
from langchain.chat_models import ChatOpenAI
from langchain.agents import load_tools, initialize_agent, AgentType

llm = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0)

# Load a specialised tool: a math LLM, not a general one
tools = load_tools(["llm-math"], llm=llm)

agent = initialize_agent(
    tools, llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True)          # False = hide the reasoning steps

print(agent.run("What is 25 multiplied by 3?"))   # → 75
```

Variation: a Wikipedia agent

```
# !pip install wikipedia
tools = load_tools(["wikipedia"], llm=llm)
agent = initialize_agent(
    tools, llm,
    agent=AgentType.STRUCTURED_CHAT_ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True)
agent.run("Who is Alan Turing and what is he famous for?")
```

TROUBLESHOOTING

- **"Could not import wikipedia python package"**: run `pip install wikipedia`, then restart the kernel. It failed on the instructor's lab but worked for some students.
- **"Agent stopped due to iteration limit or time limit"**: restart the kernel and run again.
- `llm-math` also needs the `numexpr` package installed.

PRACTICAL

L4 Lab 4 · Loading a PDF

This answers a question from the previous class. Loading a PDF works exactly like `TextLoader`, just with a different loader. The dataset is `Michael_Resume.pdf`.

```
# !pip install pypdf
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

# 1. Load + split into pages
pdf_pages = PyPDFLoader("Michael_Resume.pdf").load_and_split()
print(len(pdf_pages))
print(pdf_pages[0].page_content)

# 2. Same pipeline as text files
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(pdf_pages)

emb = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
db = FAISS.from_documents(chunks, emb)

# 3. Retrieve
docs = db.similarity_search("What are Michael's skills?", k=3)
for d in docs: print(d.page_content[:200])
```

Preview: ask a question over the PDF (the full RAG loop)

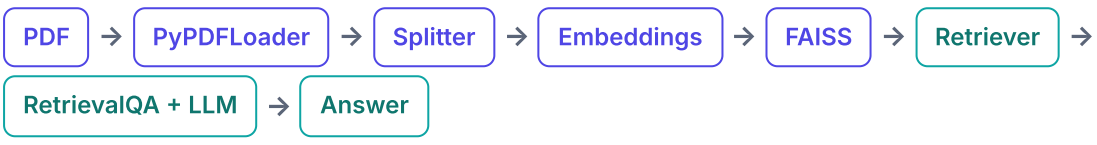
In class, Ashish asked for a demo of reading a PDF, storing it in a vector DB and running a prompt over it. The trainer will demo it next session. This is the complete pattern, building on the `db` created above:

```
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

llm = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0)
retriever = db.as_retriever(search_kwargs={"k": 3})

qa = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",          # or "refine" / "map_reduce" for big PDFs
    retriever=retriever,
    return_source_documents=True)

result = qa({"query": "Summarise Michael's work experience"})
print(result["result"])
for d in result["source_documents"]:
    print("page", d.metadata.get("page"), "→", d.page_content[:100])
```



BEFORE THE CLASS DEMO

Compare this with the trainer's version tomorrow. The lab may use slightly different imports or LangChain versions.

THEORY

7 LLM fine-tuning

Fine-tuning adapts a pretrained LLM to a **specific task or domain** by training it further on **curated, labelled data**. In ML/DL we tuned hyperparameters. With LLMs we also shape the data, prompt format, tokenization and chains. Even changing `temperature` is a light form of tuning.

> **ANALOGY: TWO IDENTICAL IPHONES**
>
> Same hardware. One owner uses default settings, while a photographer tunes the camera settings and gets sharper, better photos. Fine-tuning is configuring the same base model for your purpose.

Why fine-tune?

- **Domain-specific results:** a healthcare bot and an automobile bot can share the same base LLM, but each needs tuning for its domain.
- **Specific tasks:** turning a general "summary agent" into a "research-review summariser".
- **Efficiency:** no need to build or train a model from scratch. About 1 lakh targeted examples can specialise a model trained on millions.
- **Trade-off:** it takes time and needs **high-quality data**.

Data preparation (the most important step)

Select relevant data → Clean → Label / tag → Format (prompt template) → Augment → Train/test split → Tune

PRACTICAL

L5 Lab 5 · Preparing data for fine-tuning

Dataset: the instruction JSONL file from the Lesson 7 dataset (about 1,400 question–answer pairs).

1 Import and load the data

```
import openai, sys, os
import pandas as pd

instruction_dataset = pd.read_json("instruction_dataset.jsonl", lines=True)
print(len(instruction_dataset))
instruction_dataset.head()
```

2 Define your own prompt format and apply it to every row

```
prompt_template = """### Question:
{question}

### Answer:
{answer}"""

fine_tuning_data = []
for _, row in instruction_dataset.iterrows():
    fine_tuning_data.append({
        "text": prompt_template.format(
            question=row["question"],
            answer=row["answer"])
    })

# Verify ONE formatted example (outside the loop)
print(fine_tuning_data[0]["text"])
```

BUG FROM CLASS: ANSWERS REPEATING

The output printed the same answer again and again because (1) a closing bracket was missing after `.format(...)` and (2) `print` was **inside** the loop. Close every bracket and keep the check-print outside the loop.

3 Simple word-level tokenization

```
from collections import Counter

text = fine_tuning_data[0]["text"]
words = text.split()

# Build a vocabulary: each unique word → an integer ID
vocabulary = {word: i + 1 for i, word in enumerate(sorted(set(words)))}
token_ids = [vocabulary[word] for word in words]

print("Original text:\n", text)
print("\nToken IDs:\n", token_ids)
print(Counter(words).most_common(5))   # word frequencies
```

ERROR FROM CLASS: "UNHASHABLE TYPE: LIST"

A list was used where a single word (string) was expected. Lists can't be dictionary keys. Build the vocabulary from `set(words)` and look up one `word` at a time.

WHAT'S HAPPENING

Raw JSON → **your** consistent Q/A format → tokens with IDs. Real fine-tuning uses a model's own tokenizer (for example `AutoTokenizer` from Hugging Face), but the idea is the same. The next session completes this pipeline.

REVISION

8 Quick revision & what's next

Concept	One-line takeaway
Vector store	Permanent storage of all data as vectors: one format, small size, fast search.
Embedding	Converts data and queries into vectors (tokenization + maths).
Retriever	Fetches the most similar vectors for a query. Depends on the vector store.
LLMChain	Prompt template + LLM.
SimpleSequentialChain	Runs chains in order. Each output feeds the next input. Order matters.
Stuff / Refine / MapReduce	Small docs / polished answers / huge data.
ConversationChain + Memory	`predict()` generates replies, and `memory.buffer` shows history.
Agent	An LLM plus tools (math, Wikipedia...) that act, observe and repeat until done.
Fine-tuning	Specialise a pretrained model using curated, labelled, formatted data.

Practice checklist

- ☐ Build a 3-chain sequential pipeline (sentence → title → 3 keywords).
- ☐ Try `ConversationBufferWindowMemory(k=2)` and watch older turns drop out.
- ☐ Get the Wikipedia agent working with `pip install wikipedia`.
- ☐ Build a PDF Q&A bot with `RetrievalQA` on your own resume.
- ☐ Format a different JSON dataset into the `### Question / ### Answer` template.

TIP FROM THE TRAINER

Don't memorise model or agent names. Keep a list, or look them up on Hugging Face or the provider's site, where starter code is given. Focus on the **process** and on writing good **prompt formats**.

ANNOUNCEMENTS

- The class notebook is uploaded on the LMS under the name **"LLM applications"**.
- 3 more topics are lined up for the next session.
- Next session will include a demo of a PDF → vector DB → prompt example (see the preview in Lab 4).
- Don't forget to submit the session feedback.

Next session (Saturday, last practical class of the cohort)

- PDF → vector DB → prompt demo (extracting information from a PDF).

- Complete the fine-tuning pipeline.
- Generate an image using an agent.
- Benchmarking and evaluating LLMs.
- **Last 2 hours:** one project implemented in class, plus one explained as an assignment.
- After this, only capstone project sessions remain.